

Multi-Sensor Anomaly & Condition Monitoring Project

Abstract

This project analyzes industrial machine sensor data to detect anomalies and monitor machine health. It uses statistical techniques, SQL analysis, and machine learning models to identify abnormal patterns in temperature, vibration, pressure, energy consumption, and production data.

Introduction

Industrial machines generate large volumes of sensor data. Monitoring this data helps in identifying potential failures early. This project focuses on anomaly detection and condition monitoring to improve machine efficiency and reduce downtime.

Objectives

- Detect anomalies in sensor data
- Monitor machine health
- Build condition index
- Develop dashboards for visualization
- Support predictive maintenance

SQL Analysis

1.Objective

To analyze sensor data using SQL and identify anomalies and machine performance metrics. SQL queries are used to calculate average sensor values for each machine and plant.

The following metrics are computed:

- Average Temperature, Average Vibration, Average Pressure

These averages help identify deviations that may indicate abnormal machine behavior.

2.Anomaly Detection Using SQL (Q3)

Approach:

- Calculate **Mean (AVG)** and **Standard Deviation (STDDEV)** for each sensor:
 - Temperature
 - Vibration
 - Pressure

Output:

- Number of anomalies per:
 - Plant
 - Machine

3.Agregated View Creation (Q4)

Objective:

Create a summarized dataset for machine performance.

Output:

- Clean summarized dataset
- Machines categorized as:
 - Critical
 - Normal

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

chdb

Tables

multi_sensor_csv

Views

sensor_condition_2025

Stored Procedures

Functions

project

Tables

Views

Stored Procedures

Functions

Administration Schemas

Information

Table: multi_sensor_csv

Columns:

Timestamp text

MachineID int

Plant int

Vibration double

Temperature double

Pressure double

EnergyConsumption double

ProductionUnits int

DefectCount int

MaintenanceFlag int

Hour text

Plant_Hour text

temperature_mean double

Temperature_std_dev double

vibration_mean double

vibration_std_dev double

pressure_mean double

pressure_std_dev double

Anomaly text

Temperature Z double

Object Info Session

solved.sql

```
1 USE chdb;
2 SELECT * FROM multi_sensor_csv LIMIT 10;
3 WITH stats AS (
4     SELECT
5         Plant,
6         MachineID,
7         AVG(Temperature) AS temp_mean,
8         STDEV(Temperature) AS temp_std,
9         AVG(Vibration) AS vib_mean,
10        STDEV(Vibration) AS vib_std,
11        AVG(Pressure) AS pres_mean,
12        STDEV(Pressure) AS pres_std
13    FROM multi_sensor_csv
14    GROUP BY Plant, MachineID
15 )
16
17 SELECT
```

Result Grid

MachineID	AvgTemp	AvgVib	AvgPressure	AvgEnergyPerUnit	TotalDefects	MaintenanceCount	Conc
102	69.6024878285185	4.972847157240734	29.8885340612434	2.3830821228976657	1111	44	Norm
120	68.8752608634005	4.887120706634756	30.675365080856423	2.360485217220539	1164	32	Norm
146	69.3846405551529	4.93577579006753	30.099060447567567	2.357878235103305	1379	43	Norm
140	69.47657004244444	4.880812974240745	30.061177902617267	2.320878472158046	1216	43	Norm
114	69.5119781564281	4.966299746595242	29.87361159928593	2.2374850918838307	1207	43	Norm
144	69.20611796121368	5.015572605559369	30.12971867324537	2.2870599355399865	1118	31	Norm
117	69.10013104801587	4.911137071312573	30.043377411273003	2.308146080813108	1157	44	Norm

multi_sensor_csv 10 Result 11 sensor_condition_2025 12 x

Output

Action Output

Time Action Message Duration / Fetch

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

chdb

Tables

multi_sensor_csv

Views

sensor_condition_2025

Stored Procedures

Functions

project

Tables

Views

Stored Procedures

Functions

Administration Schemas

Information

View: sensor_condition_2025

Columns:

MachineID int

AvgTemp double

AvgVib double

AvgPressure double

AvgEnergyPerUnit double

TotalDefects decimal

MaintenanceCount decimal

ConditionBucket varchar(1)

Object Info Session

solved.sql

```
49 SELECT
50     MachineID,
51     AVG(Temperature) AS AvgTemp,
52     AVG(Vibration) AS AvgVib,
53     AVG(Pressure) AS AvgPressure,
54     AVG(EnergyConsumption / ProductionUnits) AS AvgEnergyPerUnit,
55     SUM(DefectCount) AS TotalDefects,
56     SUM(MaintenanceFlag) AS MaintenanceCount,
57     AVG(Temperature + Vibration + Pressure) AS ConditionScore
```

Result Grid

MachineID	AvgTemp	AvgVib	AvgPressure	AvgEnergyPerUnit	TotalDefects	MaintenanceCount	ConditionBucket
126	70.53939984703248	5.145819416413965	30.281474112718204	2.3471639126108546	1197	35	Critical
104	71.06557373814539	4.98226150357394	29.86123096273185	2.34533198361415	1209	39	Critical
136	70.60815671948784	5.008671265056095	30.049322615585368	2.300839675576531	1234	49	Critical
137	70.60923955972162	5.03042689153132	29.988467215730882	2.4240506234702717	1328	45	Critical
112	70.86134915764265	5.0722251094516135	29.659484166972696	2.4302607851304314	1171	35	Critical
125	70.7028360794944	4.91558567192401	29.904729978708854	2.3397480171552643	1166	41	Normal
135	70.31374872874076	4.993069489567903	30.18230106365432	2.3252181765110913	1261	42	Normal
103	70.3060609390478	5.03934056383335	30.12490147047653	2.4141279343061237	1264	42	Normal
115	70.4716266612746	4.944763772627452	30.01925368384806	2.4227802231668263	1261	36	Normal
119	70.56375777478806	4.986992664963915	29.871992740747434	2.331071263066634	1124	37	Normal

multi_sensor_csv 4 Result 5 sensor_condition_2025 6 x

Output

Action Output

Time Action Message Duration / Fetch

9	19:53:28	SELECT * FROM sensor_condition_2025 LIMIT 0, 1000	50 row(s) returned	0.078 sec / 0.000 sec
10	19:53:46	USE chdb	0 row(s) affected	0.000 sec
11	19:53:46	SELECT * FROM multi_sensor_csv LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec
12	19:53:46	WITH stats AS (SELECT Plant, MachineID, AVG(Temperature) AS temp_mean, STDEV(Temperature) AS temp_std, AVG(Vibration) AS vib_mean, STDEV(Vibration) AS vib_std, AVG(Pressure) AS pres_mean, STDEV(Pressure) AS pres_std	89 row(s) returned	0.219 sec / 0.000 sec
13	19:53:46	CREATE OR REPLACE VIEW sensor_condition_2025 AS SELECT MachineID, AvgTemp, AvgVib, AvgPressure, AvgEnergyPerUnit, TotalDefects, MaintenanceCount, ConditionBucket	0 row(s) affected	0.016 sec
14	19:53:46	SELECT * FROM sensor_condition_2025 LIMIT 0, 1000	50 row(s) returned	0.062 sec / 0.000 sec

Python Analysis

1.Objective:

Detect anomalies using Machine Learning.The data-set is imported into Python using the Visual Studio Code environment.Using the Pandas library, the data-set is loaded and inspected to understand its structure, columns, and data types.

2.Anomaly Detection Using Python (Q5)

Method Used:

- Isolation Forest Algorithm

Steps:

1. Load dataset using Pandas
2. Select relevant features:
 1. Temperature
 2. Vibration
 3. Pressure
 4. Energy Consumption
 5. Production Units
3. Train Isolation Forest model
4. Predict anomalies

Output:

- List of top 200 anomalies
- Includes:
 - Machine ID
 - Timestamp

3.Health Score Prediction (Q6)

Objective:

Predict machine health using regression.

Steps:

1. Create HealthScore column
2. Train regression model
3. Evaluate model performance

Output:

- Predicted health scores
- Identification of key influencing factors

The screenshot shows a VS Code editor with a Python script named `python final Q 5.6.py`. The script imports `IsolationForest`, `train_test_split`, `LinearRegression`, and `mean_squared_error`, `r2_score`. It reads data from `multi_sensor_data.csv` and performs a data preview. The output in the terminal shows a data preview with columns: `Timestamp`, `MachineID`, `Plant`, `Temperature`, `Vibration`, `Anomaly`, `Temperature Z`, `Vibration Z`, `Pressure Z`, and `Condition`. The data is organized into a table with 10 columns and 10 rows of data.

```
python final Q 5.6.py
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# STEP 1: Load Data
df = pd.read_csv("multi_sensor_data.csv")

print("Data Preview:")
print(df.head())

# Q5: ANOMALY DETECTION

features = ["Temperature", "Vibration", "Pressure", "EnergyConsumption", "ProductionUnits"]
X = df[features]
features = ["Temperature", "Vibration", "Pressure", "EnergyConsumption", "ProductionUnits"]
```

Timestamp	MachineID	Plant	Temperature	Vibration	Anomaly	Temperature Z	Vibration Z	Pressure Z	Condition
01-01-2025 00:00	135	Plant_B	70.368890	5.924252	Normal	0.105937	0.674498	0.032646	0.2
01-01-2025 01:00	114	Plant_A	59.632304	5.408591	Normal	-1.040064	0.257354	-0.983755	-0.5
01-01-2025 02:00	147	Plant_C	71.513375	4.277121	Normal	0.066663	-0.465971	0.809588	0.1
01-01-2025 03:00	138	Plant_B	84.742846	4.013507	Normal	1.635126	-0.585449	1.498791	0.8
01-01-2025 04:00	116	Plant_C	61.904598	3.402422	Normal	-0.722495	-1.084393	-1.021155	-0.9

The screenshot shows a VS Code editor with a CSV file named `top_200_anomalies.csv`. The data is organized into a table with 10 columns: `Timestamp`, `MachineID`, `Plant`, `Temperature`, `Vibration`, `Pressure`, `EnergyConsumption`, `ProductionUnits`, `DefectCount`, and `Condition`. The data is organized into a table with 10 columns and 10 rows of data.

Timestamp	MachineID	Plant	Temperature	Vibration	Pressure	EnergyConsumption	ProductionUnits	DefectCount	Condition
01-01-2025 03:00	138	Plant_B	84.74284581	4.01350746	37.43134639	177.0032425	75.3,0,3	Plant_B_3,69.0243107	
01-01-2025 05:00	109	Plant_C	89.65419232	5.274782587	38.7856345	195.2560809	129.5,1,5	Plant_C_5,70.674301	
01-01-2025 07:00	136	Plant_B	80.85106537	4.739044997	27.87336527	310.7357953	187.3,0,7	Plant_B_7,69.4595	
01-01-2025 08:00	139	Plant_B	55.2672174	4.581760248	15.88107805	253.0597544	161.4,1,8	Plant_B_8,69.98509	
01-01-2025 11:00	130	Plant_B	70.56715784	7.795982642	21.89346336	220.526233	136.4,1,11	Plant_B_11,69.327	
01-01-2025 16:00	149	Plant_C	77.54246271	8.075543243	20.81465666	271.9808592	142.3,0,16	Plant_C_16,70.99	
01-01-2025 18:00	116	Plant_B	59.80820224	2.11733984	29.65249265	309.2065157	140.3,1,18	Plant_B_18,69.494	
01-01-2025 19:00	127	Plant_B	46.82966527	7.397077351	27.89857146	211.152071	188.5,1,19	Plant_B_19,71.061	
01-01-2025 22:00	111	Plant_A	52.01578153	4.531025981	24.08971545	218.6635964	160.2,0,22	Plant_A_22,69.92	
01-01-2025 23:00	101	Plant_C	49.19907974	3.040957861	24.56551517	240.4683793	74.3,0,23	Plant_C_23,69.853	
02-01-2025 10:00	118	Plant_A	93.97282708	7.362820418	33.74372497	278.1283817	57.1,0,10	Plant_A_10,70.675	
02-01-2025 11:00	139	Plant_B	68.6484881	2.392947209	27.82300725	252.6064651	57.1,0,11	Plant_B_11,69.3275	
02-01-2025 13:00	114	Plant_B	74.74440073	1.98199311	24.09371002	250.2660966	179.3,0,13	Plant_B_13,69.894	
02-01-2025 16:00	116	Plant_B	72.34688015	7.648111598	34.86613584	311.0616155	92.4,0,16	Plant_B_16,70.408	
02-01-2025 17:00	104	Plant_B	78.03453725	3.653889342	30.16270144	322.1983914	56.0,0,17	Plant_B_17,71.012	
02-01-2025 18:00	146	Plant_A	78.09635854	3.850379349	32.49957083	346.9558352	113.3,0,18	Plant_A_18,69.80	
02-01-2025 20:00	141	Plant_B	76.96072058	7.525955993	31.17954024	288.3279224	197.1,0,20	Plant_B_20,69.17	
02-01-2025 21:00	127	Plant_A	52.05649629	7.312671116	26.38761754	278.7620138	165.5,0,21	Plant_A_21,69.22	
03-01-2025 07:00	107	Plant_B	68.21022506	7.167817231	26.96304941	161.2970101	104.4,0,7	Plant_B_7,69.4595	
03-01-2025 08:00	136	Plant_C	85.83443155	6.345282344	29.09053708	339.9471335	150.2,0,8	Plant_C_8,70.0406	
03-01-2025 17:00	127	Plant_C	92.08189558	6.202798997	32.36586434	262.8873249	84.3,0,17	Plant_C_17,69.742	
03-01-2025 21:00	133	Plant_C	71.84598464	5.916002853	34.01621719	171.3876276	185.0,0,21	Plant_C_21,71.12	
03-01-2025 23:00	121	Plant_C	64.4566407	8.859658382	32.52006019	207.6081643	100.2,0,23	Plant_C_23,69.853	
04-01-2025 03:00	114	Plant_B	66.69526666	6.281983747	25.94318194	357.192304	66.3,0,3	Plant_B_3,69.0243107	
04-01-2025 05:00	131	Plant_A	71.04474582	7.835783289	30.85726151	296.4705814	79.2,0,5	Plant_A_5,70.49812	
04-01-2025 07:00	138	Plant_B	46.09299927	4.528513924	35.59163992	257.0569252	171.4,0,7	Plant_A_7,69.2767	
04-01-2025 14:00	113	Plant_C	63.73663813	5.783338613	39.53285739	277.9579798	60.3,0,14	Plant_C_14,70.020	
04-01-2025 17:00	131	Plant_C	75.03874605	1.325851436	28.18611957	197.5181128	194.0,0,17	Plant_C_17,69.742	
04-01-2025 22:00	114	Plant_C	53.97423597	4.319996327	30.42612737	183.5311281	171.4,0,22	Plant_C_22,70.13	
04-01-2025 23:00	147	Plant_B	69.38239759	4.582150826	32.32578425	145.728348	185.4,0,23	Plant_B_23,70.562	
05-01-2025 01:00	118	Plant_A	66.36759904	4.14337466	38.70789599	332.9171658	62.4,0,1	Plant_A_1,70.067984	
05-01-2025 09:00	126	Plant_B	54.83676962	7.76661441	36.23153072	297.0390654	156.2,0,9	Plant_B_9,70.01190	
05-01-2025 12:00	130	Plant_C	65.74940217	7.821940998	20.81302592	277.4497413	77.2,1,12	Plant_C_12,69.631	
05-01-2025 14:00	105	Plant_B	59.65915986	4.620921533	22.56616883	258.1327091	194.4,0,14	Plant_B_14,70.13	
05-01-2025 15:00	141	Plant_C	71.47443128	2.85178771	33.47623218	216.6095605	191.4,0,15	Plant_C_15,69.432	
05-01-2025 17:00	105	Plant_B	60.75546708	5.509642494	20.34124162	292.8667792	71.0,0,17	Plant_B_17,71.012	

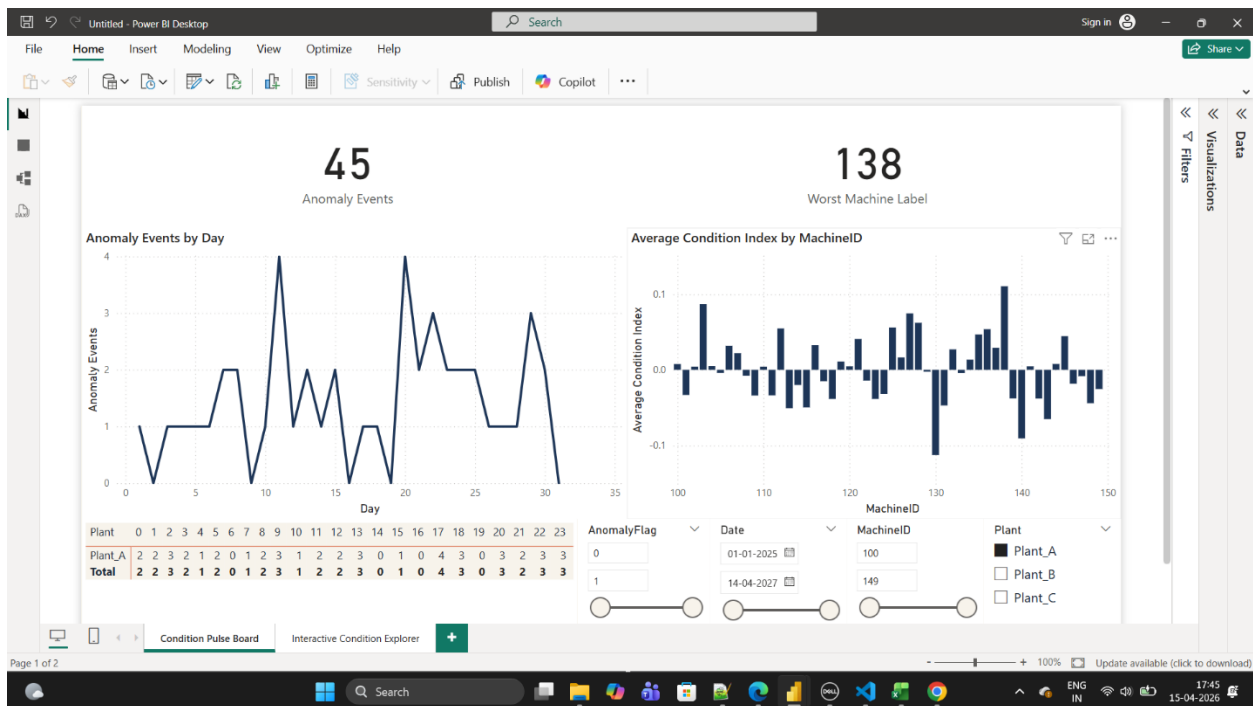
Sensor Monitoring Dashboard

The processed dataset is loaded into Power BI, where interactive visualizations are created to provide insights into machine health.

Key components of the dashboard include:

- **Anomaly Count KPI** to display the total number of detected anomalies
- **Trend Analysis** showing anomalies over time (daily basis)
- **Machine Ranking** based on Condition Index to identify poorly performing machines
- **Slicers/Filters** for:
 - Plant ID
 - Machine ID

This dashboard helps users quickly identify critical machines and monitor system performance effectively.



Interactive Condition Analysis

The dashboard includes synchronized visualizations for:

- Temperature trends
- Vibration patterns
- Pressure variations

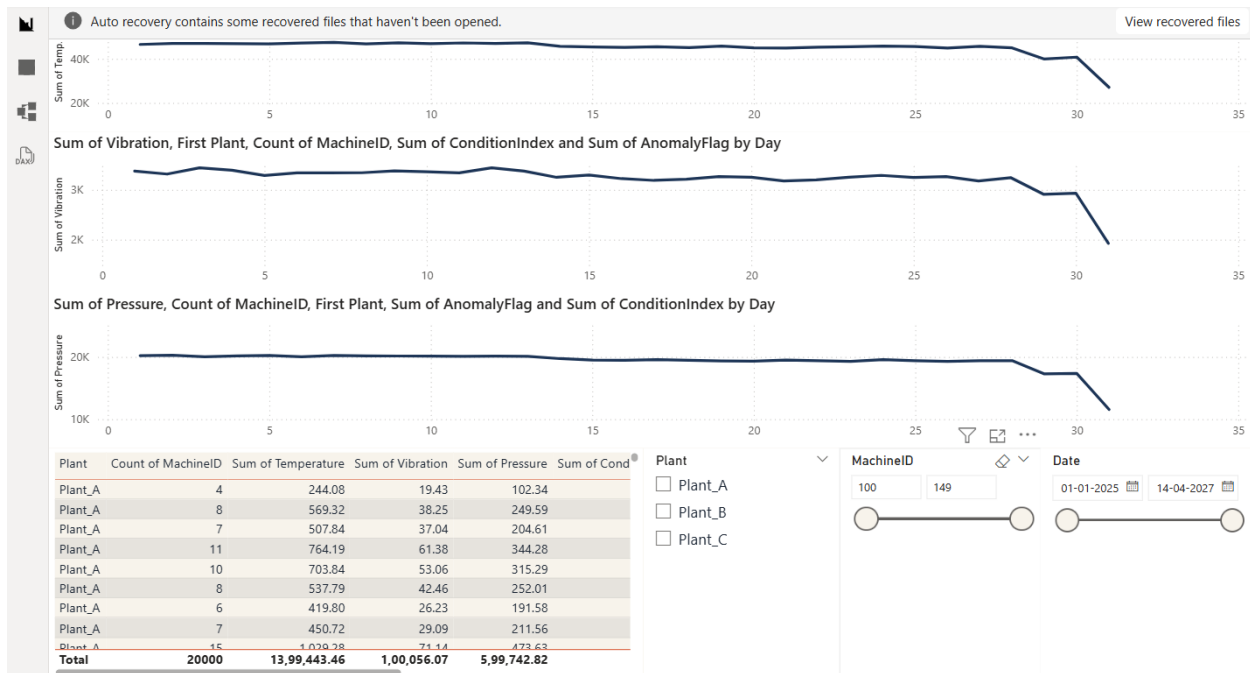
Anomaly points are highlighted on the charts to clearly show abnormal behavior over time.

Additional features include:

- A detailed table displaying anomaly events
- Time-based analysis for deeper insights
- Interactive filtering for better exploration

This allows users to:

- Analyze sensor behavior over time
- Detect patterns and irregularities
- Support data-driven maintenance decisions



Project Outcome

This project shows how we can use data analysis and machine learning to understand machine sensor data and find unusual behavior (anomalies).

Instead of checking machines manually, the system automatically monitors their performance and alerts us when something is wrong.

Main Results:

- The system can **automatically detect problems** in machines
- It helps to **check the health of machines** easily
- Dashboards make it **easy to see and understand data**
- Decisions are based on **data, not guesswork**

Overall Benefit:

This helps companies fix issues **before machines fail**, reduce downtime, and improve efficiency in industrial operations.